

End-to-End Task Workflow

End-to-End Task Workflow

This tab gives the high-level lifecycle of a task in Agentic Code QA. The detailed instructions for writing each artifact live in the next four tabs.



Workflow

① Writers create a repo-grounded task.

Writers claim an assigned repo, explore the codebase at the assigned commit, choose a realistic engineering question, write the prompt and golden answer, create the rubric, and run difficulty calibration + model evaluation.

② Reviewers inspect the task.

Reviewers check whether the prompt is fair and specific, the golden answer is correct, the rubric is measurable, the repository context is valid, and the difficulty tag matches the real task burden.

③ Writers revise when needed.

If the reviewer sends the task back, writers should address review feedback before claiming or starting new tasks. First tasks may take multiple revision cycles, but long-term quality is measured by low revision count and clean submissions.

④ Approved tasks are finalized.

Once approved, the task is finalized (“merged”) and ready for downstream use. The writer-facing model eval/calibration step already happened before submission as part of task creation.

Required Components Per Task

Each task must include the following core artifacts. Use this table as a quick reference; the detailed writing rules are in the later workflow tabs.

Artifact	Purpose	Key Requirements
Prompt (prompt_statement.md)	The question shown to the agent.	Contains only the question. May reference files, modules, functions, symptoms, patches, or line numbers by name. Must not paste large code snippets, extracted docs, or answer hints.
Golden Answer (golden_answer.md)	The expert reference answer.	Fully answers the prompt from repo evidence. Names relevant files/functions/flows and explains the reasoning behind the conclusion. Should be specific enough to produce atomic rubric criteria.
Rubric (rubric.json)	The grading rubric for model answers.	Usually 15-35 total criteria; up to 40 for hard tasks when needed. Include reasoning + completeness criteria, 5-10 penalties, and at least 2 style criteria. Never mark style as critical or penalty.
pr.patch (only some intents)	PR/change artifact provided for some intents.	Required when the question depends on a prewritten proposed code change.

		Examples: PR triage, architecture/design questions involving a patch, or code onboarding questions about changed behavior.
--	--	--